

Clasificación de gatos y perros mediante redes neuronales convolucionales y prueba de funcionamiento en Roboflow

Andrea Carolina Roper Lozano
Maria Isabel Posada Niasa

18 de junio de 2026

Proyecto TAM
Universidad Nacional de Colombia

Índice

1. Introducción	3
2. Objetivo general	3
3. Objetivos específicos	3
4. Flujo general del proyecto	4
5. Descripción del dataset	4
6. Preprocesamiento de imágenes	5
7. Modelo 1: CNN básica	5
7.1. Arquitectura de la CNN	6
7.2. Función de pérdida	6
7.3. Entrenamiento	6
8. Evaluación de la CNN básica	7

9. Modelo 2: MobileNetV2 con transferencia de aprendizaje	7
9.1. Justificación del uso de transferencia de aprendizaje	7
9.2. Arquitectura utilizada	8
10.Evaluación de MobileNetV2	8
10.1. Interpretación de las curvas de entrenamiento	8
11.Métricas utilizadas	9
12.Comparación de modelos	9
13.Prueba con imagen individual	9
14.Preparación del dataset reducido para Roboflow	10
15.Prueba en Roboflow	10
16.Conclusiones	11
17.Trabajo futuro	11

1. Introducción

El presente proyecto desarrolla un sistema de visión por computador para clasificar imágenes de mascotas en dos clases principales: `cat` y `dog`. El trabajo se realizó principalmente en Google Colab, donde se entrenaron y evaluaron modelos de clasificación basados en redes neuronales convolucionales.

El proyecto compara dos enfoques: una red neuronal convolucional básica entrenada desde cero y un modelo basado en transferencia de aprendizaje con MobileNetV2. La finalidad es analizar cómo cambia el desempeño cuando se utiliza una arquitectura previamente entrenada sobre una gran base de datos de imágenes.

Adicionalmente, se preparó una versión reducida del dataset en formato YOLO para realizar una prueba en Roboflow. Esta etapa no corresponde al núcleo principal del proyecto, sino a una verificación práctica de que la base de datos podía organizarse correctamente para detección y que las clases `cat` y `dog` podían ser reconocidas dentro de la plataforma.

2. Objetivo general

Desarrollar y evaluar un modelo de clasificación de imágenes capaz de diferenciar entre gatos y perros, utilizando redes neuronales convolucionales en Google Colab y validando de forma complementaria el funcionamiento de la base de datos en Roboflow.

3. Objetivos específicos

- Preparar el dataset Oxford Pets para trabajar con dos clases principales: `cat` y `dog`.
- Implementar una CNN básica para clasificación binaria de imágenes.
- Aplicar transferencia de aprendizaje con MobileNetV2.
- Comparar el rendimiento de la CNN básica frente a MobileNetV2.
- Evaluar los modelos mediante métricas de clasificación: accuracy, precision, recall, F1-score y matriz de confusión.
- Construir una versión reducida del dataset en formato YOLO para verificar su funcionamiento en Roboflow.
- Interpretar los resultados obtenidos y establecer conclusiones del proyecto.

4. Flujo general del proyecto

El proyecto se organizó en dos fases. La primera fase corresponde al desarrollo principal en Google Colab, donde se realizó la clasificación de imágenes. La segunda fase corresponde a una prueba complementaria en Roboflow para validar que el dataset reducido funcionara correctamente en un entorno de detección.

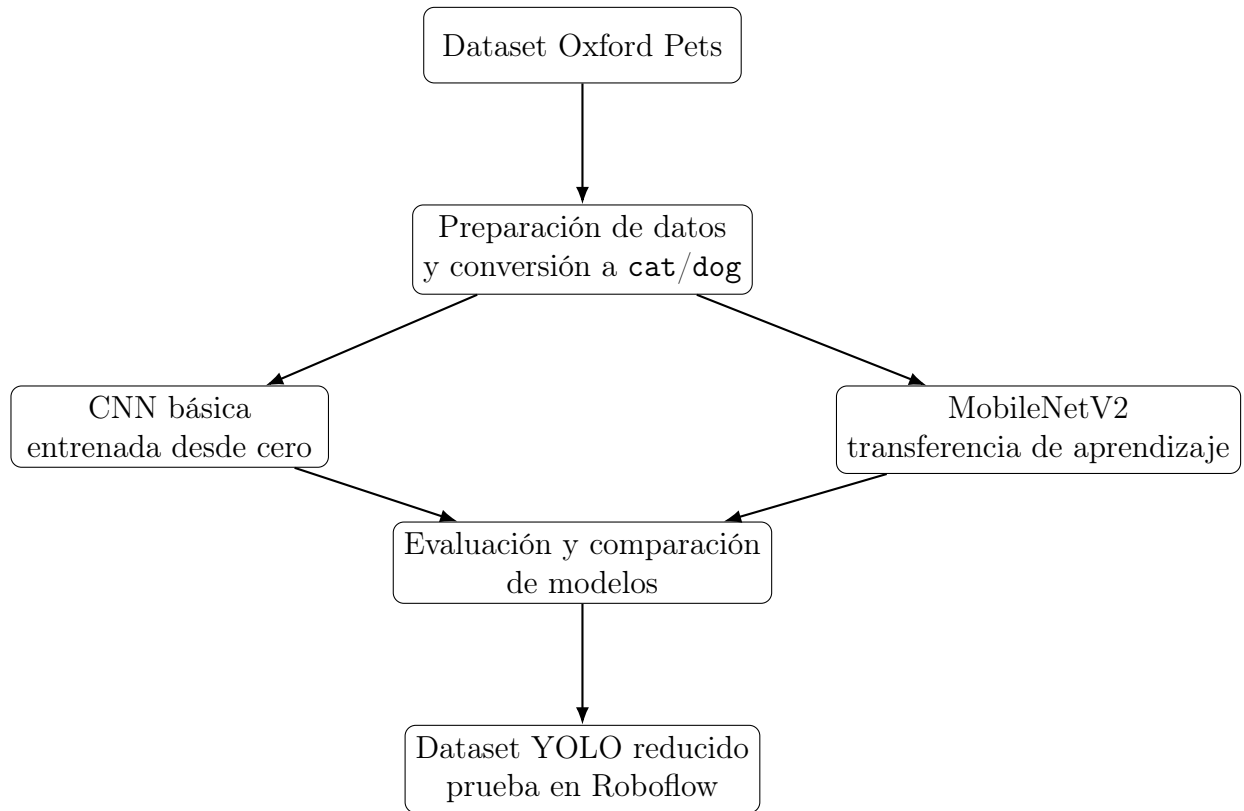


Figura 1: Flujo general del proyecto.

5. Descripción del dataset

La base de datos utilizada corresponde a Oxford Pets, descargada desde Roboflow en formato YOLOv8. Originalmente, el dataset contiene clases asociadas a diferentes razas de gatos y perros. Para este proyecto, dichas clases se agruparon en dos categorías generales:

- Clase 0: cat
- Clase 1: dog

En total se identificaron 37 clases originales: 12 razas de gatos y 25 razas de perros. Estas clases fueron transformadas a un problema de clasificación binaria.

Tabla 1: Distribución general del dataset después de la conversión binaria.

Partición	Cat	Dog
Entrenamiento	4070	8809
Validación	249	487
Prueba	125	243
Total	4444	9539

Se observa un desbalance entre las clases, debido a que existen más imágenes de perros que de gatos. Por esta razón, durante el entrenamiento se utilizaron pesos de clase para compensar este comportamiento.

6. Preprocesamiento de imágenes

Para entrenar los modelos en Colab, las imágenes fueron cargadas desde sus rutas, convertidas a formato RGB, redimensionadas a 128×128 píxeles y transformadas a tensores de TensorFlow.

En la CNN básica, los valores de píxeles se normalizaron en el rango $[0, 1]$. En MobileNetV2 se utilizó el preprocesamiento propio de la arquitectura mediante la función `preprocess_input`, debido a que esta red fue previamente entrenada bajo una normalización específica.

El flujo de preparación de datos fue el siguiente:

1. Lectura de la imagen desde la ruta almacenada en el DataFrame.
2. Conversión a imagen RGB.
3. Redimensionamiento a 128×128 píxeles.
4. Normalización o preprocesamiento según el modelo.
5. Agrupamiento en lotes mediante `batch`.
6. Optimización de lectura mediante `prefetch`.

7. Modelo 1: CNN básica

El primer modelo implementado fue una red neuronal convolucional básica entrenada desde cero. Su objetivo fue servir como modelo base para comparar contra MobileNetV2.

7.1. Arquitectura de la CNN

La arquitectura se compone de bloques convolucionales encargados de extraer características visuales de las imágenes, seguidos por capas densas para realizar la clasificación final.

Tabla 2: Arquitectura general de la CNN básica.

Capa	Función
Entrada $128 \times 128 \times 3$	Recibe imagen RGB.
Aumentación de datos	Aplica rotaciones, zoom, contraste y volteo horizontal.
Conv2D + ReLU	Extrae patrones locales de la imagen.
MaxPooling2D	Reduce la dimensión espacial de los mapas de características.
Bloques Conv2D adicionales	Aprenden características visuales más complejas.
Flatten	Convierte los mapas de características en un vector.
Dense + ReLU	Aprende combinaciones no lineales de las características.
Dropout	Reduce el riesgo de sobreajuste.
Dense + Sigmoid	Genera la probabilidad de pertenecer a la clase dog .

La salida del modelo se interpreta de la siguiente manera:

$$\hat{y} = \begin{cases} 0, & \text{si } p(\text{dog}) < 0,5 \Rightarrow \text{cat}, \\ 1, & \text{si } p(\text{dog}) \geq 0,5 \Rightarrow \text{dog}. \end{cases}$$

7.2. Función de pérdida

Como el problema es de clasificación binaria, se utilizó la función de pérdida *binary crossentropy*:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

donde y es la etiqueta real y $\hat{y}$ es la probabilidad estimada por el modelo.

7.3. Entrenamiento

El modelo se entrenó usando el optimizador Adam, con una tasa de aprendizaje de 10^{-4} . También se emplearon dos mecanismos de control:

- **EarlyStopping:** detiene el entrenamiento si la pérdida de validación deja de mejorar.
- **ModelCheckpoint:** guarda automáticamente la mejor versión del modelo según la pérdida de validación.

8. Evaluación de la CNN básica

La CNN básica obtuvo un desempeño moderado. El modelo logró aprender patrones generales para diferenciar gatos y perros, aunque presentó confusiones en algunas imágenes.

Tabla 3: Reporte de clasificación de la CNN básica.

Clase	Precision	Recall	F1-score	Support
Cat	0.63	0.68	0.65	125
Dog	0.83	0.79	0.81	243
Accuracy	0.76			

La matriz de confusión obtenida fue:

$$\begin{bmatrix} 85 & 40 \\ 50 & 193 \end{bmatrix}$$

Esto indica que el modelo clasificó correctamente 85 gatos y 193 perros. Sin embargo, confundió 40 gatos como perros y 50 perros como gatos.

9. Modelo 2: MobileNetV2 con transferencia de aprendizaje

El segundo modelo utilizó transferencia de aprendizaje con MobileNetV2. Esta arquitectura ya fue entrenada previamente sobre ImageNet, por lo que posee filtros capaces de detectar patrones visuales generales como bordes, texturas, formas y estructuras.

9.1. Justificación del uso de transferencia de aprendizaje

Entrenar una red profunda desde cero requiere grandes cantidades de datos y alto costo computacional. En cambio, con transferencia de aprendizaje se aprovecha una red previamente entrenada y se adapta al problema específico. En este caso, MobileNetV2 se usó como extractor de características y se agregó un clasificador binario propio para las clases `cat` y `dog`.

9.2. Arquitectura utilizada

Tabla 4: Arquitectura del modelo MobileNetV2 utilizado.

Componente	Función
Entrada $128 \times 128 \times 3$	Imagen RGB preprocesada para MobileNetV2.
MobileNetV2 sin capa superior	Extrae características visuales.
Congelamiento de pesos	Evita modificar la base preentrenada.
GlobalAveragePooling2D	Resume los mapas de características.
Dense(128) + ReLU	Aprende la separación entre gatos y perros.
Dropout(0.4)	Reduce sobreajuste.
Dense(1) + Sigmoid	Entrega la probabilidad de clase dog .

La base convolucional de MobileNetV2 se mantuvo congelada, de modo que el entrenamiento se concentró principalmente en las capas finales agregadas.

10. Evaluación de MobileNetV2

MobileNetV2 presentó un rendimiento significativamente superior a la CNN básica. En el conjunto de prueba alcanzó una exactitud aproximada de 98.6 %.

Tabla 5: Reporte de clasificación de MobileNetV2.

Clase	Precision	Recall	F1-score	Support
Cat	0.98	0.98	0.98	125
Dog	0.99	0.99	0.99	243
Accuracy	0.99			

La matriz de confusión obtenida fue:

$$\begin{bmatrix} 122 & 3 \\ 2 & 241 \end{bmatrix}$$

Esto significa que el modelo clasificó correctamente 122 gatos y 241 perros, cometiendo únicamente 5 errores sobre 368 imágenes de prueba.

10.1. Interpretación de las curvas de entrenamiento

Las curvas de entrenamiento mostraron que MobileNetV2 aprendió rápidamente. La pérdida de entrenamiento disminuyó de forma continua y la exactitud aumentó hasta valores cercanos

a 1.0. La validación se mantuvo estable, con una exactitud cercana a 0.98 y una pérdida baja.

Aunque se observa una ligera separación entre entrenamiento y validación, esto no representa un problema grave, ya que el desempeño de validación se mantuvo alto y estable. En general, el modelo generalizó adecuadamente sobre imágenes no vistas.

11. Métricas utilizadas

Para evaluar los modelos se utilizaron las siguientes métricas:

- **Accuracy:** mide el porcentaje total de predicciones correctas.
- **Precision:** mide cuántas predicciones de una clase fueron realmente correctas.
- **Recall:** mide cuántos ejemplos reales de una clase fueron detectados correctamente.
- **F1-score:** combina precision y recall en una sola métrica.
- **Matriz de confusión:** permite observar aciertos y errores por clase.

Estas métricas son importantes porque no basta con revisar la exactitud global; también es necesario analizar si el modelo reconoce correctamente ambas clases.

12. Comparación de modelos

Tabla 6: Comparación general entre la CNN básica y MobileNetV2.

Modelo	Accuracy	Precision	Recall	F1-score
CNN básica	0.76	0.83	0.79	0.81
MobileNetV2	0.99	0.99	0.99	0.99

Los resultados muestran que MobileNetV2 supera ampliamente a la CNN básica. Esto se debe a que MobileNetV2 aprovecha características visuales previamente aprendidas en ImageNet, mientras que la CNN básica debe aprender todos los filtros desde cero con el dataset disponible.

13. Prueba con imagen individual

Después de evaluar el modelo MobileNetV2, se realizó una prueba con una imagen cargada manualmente desde el computador. Esta imagen fue preprocesada con el mismo procedimiento utilizado durante el entrenamiento de MobileNetV2.

El modelo generó una probabilidad de pertenecer a la clase `dog`. Si dicha probabilidad era mayor o igual a 0.5, la imagen se clasificaba como perro; si era menor, se clasificaba como gato.

Esta prueba permitió verificar el funcionamiento del modelo sobre una imagen externa al flujo de entrenamiento y evaluación.

14. Preparación del dataset reducido para Roboflow

Aunque el objetivo principal del proyecto fue la clasificación en Colab, también se preparó una versión reducida del dataset en formato YOLO para realizar una prueba de funcionamiento en Roboflow.

Inicialmente, el dataset completo era demasiado grande para subirlo de forma práctica. Por esta razón, se construyó una versión reducida y balanceada:

Tabla 7: Distribución del dataset reducido para Roboflow.

Partición	Cat	Dog
Entrenamiento	600	600
Validación	150	150
Prueba	100	100
Total	850	850

Las imágenes fueron redimensionadas a 416×416 píxeles para reducir el peso del archivo. Las etiquetas YOLO originales fueron modificadas para reemplazar las clases de raza por las clases binarias:

$$0 \rightarrow \text{cat}, \quad 1 \rightarrow \text{dog}.$$

Las coordenadas de las cajas delimitadoras no se modificaron, ya que en YOLO están normalizadas entre 0 y 1. Finalmente, el dataset reducido fue comprimido en un archivo `.zip` de aproximadamente 44.10 MB.

15. Prueba en Roboflow

En Roboflow se realizó una prueba complementaria para verificar que la base de datos reducida funcionara correctamente en un entorno de detección de objetos. La finalidad de esta etapa fue comprobar que:

- Las imágenes se cargaran correctamente.

- Las etiquetas `cat` y `dog` fueran reconocidas por la plataforma.
- Las anotaciones en formato YOLO estuvieran asociadas correctamente a las imágenes.
- La base de datos permitiera realizar detecciones de prueba.

Es importante aclarar que el núcleo del proyecto fue la clasificación realizada en Colab. Roboflow se utilizó únicamente como una validación práctica del dataset en formato de detección y como prueba visual de funcionamiento.

16. Conclusiones

1. El proyecto permitió construir un sistema de clasificación binaria para diferenciar imágenes de gatos y perros.
2. La CNN básica logró aprender patrones visuales generales, alcanzando una exactitud aproximada de 76 %, pero presentó errores relevantes entre ambas clases.
3. MobileNetV2 obtuvo un desempeño superior, con una exactitud cercana al 99 % en el conjunto de prueba.
4. La transferencia de aprendizaje fue la estrategia más efectiva, ya que permitió aprovechar características visuales previamente aprendidas en ImageNet.
5. Las métricas de clasificación y las matrices de confusión evidenciaron que MobileNetV2 reconoció correctamente la mayoría de imágenes de ambas clases.
6. La prueba en Roboflow permitió verificar que el dataset reducido en formato YOLO estaba correctamente organizado y que las clases `cat` y `dog` podían ser utilizadas en una tarea de detección.
7. En general, el proyecto demuestra la utilidad de las redes convolucionales y de la transferencia de aprendizaje para resolver problemas de clasificación de imágenes.

17. Trabajo futuro

Como posibles mejoras del proyecto se proponen:

- Probar otras arquitecturas preentrenadas, como EfficientNet, ResNet o VGG.
- Aplicar ajuste fino desbloqueando algunas capas finales de MobileNetV2.
- Aumentar el tamaño del dataset de entrenamiento.
- Evaluar el modelo con imágenes tomadas en condiciones reales.

- Desarrollar una aplicación sencilla para cargar imágenes y obtener predicciones automáticamente.
- Entrenar formalmente un detector en Roboflow si se desea extender el proyecto desde clasificación hacia detección de objetos.